

Milestone 2. Core functionalities Development

Activity 2.1. Develop the core functionalities

Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

Activity 2.1 Develop the core functionalities:

Workout Plan Generation

Function: `generate_workout_gemini()`

File: `gemini_generator.py`

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to the Gemini 1.5 Pro model, which returns a structured, day-wise plan including:

Warm-up (5–10 mins)

Main workout (exercise details, sets, reps)

Cooldown or recovery tip

The output is returned to `routes.py` and rendered on the `result.html` page using `<pre>` blocks for formatting.

Function to generate workout

```
def generate_workout_gemini(user_input):
```

```
    prompt = f"""
```

```
    You are a professional fitness trainer.
```

```
    Create a personalized, structured 7-day workout plan for someone with the goal of
```

```
    **{user_input['goal']}**, and prefers **{user_input['intensity']}**
```

```
    intensity workouts.
```

```
    Each day must include:
```

```
    - A warm-up (5-10 mins)
```

- Main workout (targeted exercises, sets & reps)
- Cooldown or recovery tip

Format:

Day 1:

Warm-up: ...

Main Workout: ...

Cooldown: ...

(Repeat for Day 2-7)

"""

try:

 response = model.generate_content(prompt)

 return response.text

except Exception as e:

 return f"Error: {e}"

Nutrition Tip Generation

Function: generate_nutrition_tip_with_flash()

File: gemini_flash_generator.py

This function generates a concise, practical nutrition or recovery tip tailored to the user's selected goal. It utilizes the Gemini Flash model to deliver a fast, helpful response such as hydration advice, protein suggestions, or general recovery best practices. This tip is displayed beneath the workout plan in result.html.

Def generate_nutrition_tip_with_flash(goal: str) -> str:

 """

 Generate a nutrition or recovery tip using Gemini Flash based on the user's fitness goal.

Args:

Goal (str): User's fitness goal – “weight loss”, “muscle gain”, or “general fitness”.

Returns:

Str: Generated tip.

“””

Prompt = (

F”Give one clear, helpful nutrition or recovery tip for someone focused on ‘{goal}’. “

“The tip should be practical, friendly, and easy to understand.”

)

Try:

Response = model.generate_content(prompt)

Return response.text.strip()

Except Exception as e:

Return f”Error generating tip: {str(e)}”

Feedback-Based Plan Updating

Function: update_workout_plan()

File: updated_plan.py

This function enables the user to revise their original plan by submitting feedback (e.g., "Add yoga", "Include more cardio"). The original plan and feedback are passed to Gemini 1.5 Pro, which returns a modified plan with the suggested updates. The updated plan is then stored and displayed alongside the new nutrition tip.

Def update_workout_plan(original_plan: str, user_feedback: str) -> str:

“””

Use Gemini 1.5 Pro to update the workout plan based on user feedback.

“””

Prompt = f”””

You are a professional fitness trainer assistant.

Here’s the original 7-day workout plan:

{original_plan}

User Feedback:

“{user_feedback}”

Based on the feedback, revise the relevant parts of the workout plan. Keep the format and rest of the plan unchanged if not needed.

“””

Try:

Response = model.generate_content(prompt)

Return response.text.strip()

Except Exception as e:

Return f”Error updating plan: {e}”

User & Plan Storage

Functions:

Save_user(), save_plan(), update_plan()

Get_original_plan(), get_user()

File: database.py

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved as a separate field to preserve both versions (original and updated).

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
```

```
    db = SessionLocal()
```

```
    existing = db.query(User).filter_by(id=user_id).first()
```

```
    if existing:
```

```
        # Update existing user info
```

```
        existing.name = name
```

```
        existing.age = age
```

```
        existing.weight = weight
```

```
        existing.goal = goal
```

```
        existing.intensity = intensity
```

```
    else:
```

```
        # Create a new user
```

```
        user = User(
```

```
            id=user_id,
```

```
            name=name,
```

```
            age=age,
```

```
            weight=weight,
```

```
            goal=goal,
```

```
            intensity=intensity,
```

```
            schedule=7 # default schedule or logic
```

```
        )
```

```
        db.add(user)
```

```
    db.commit()
```

```
db.close()
```

```
# stores the plan in the database
```

```
def save_plan(user_id: int, plan: str):
```

```
    db = SessionLocal()
```

```
    workout = WorkoutPlan(user_id=user_id, original_plan=plan)
```

```
    db.add(workout)
```

```
    db.commit()
```

```
    db.close()
```

```
# Update the plan based on feedback
```

```
def update_plan(user_id: int, updated_text: str):
```

```
    Db = SessionLocal()
```

```
    Workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
```

```
    if workout:
```

```
        Workout.updated_plan = updated_text
```

```
        Db.commit()
```

```
    Db.close()
```

```
# Fetch the original plan for a user
```

```
def get_original_plan(user_id: int):
```

```
    Db = SessionLocal()
```

```
    Plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user_id).first()
```

```
    return plan.original_plan if plan else None
```

```
Def get_user(user_id: int):
```

```
    Db = SessionLocal()
```

```
    Return db.query(User).filter(User.id == user_id).first()
```

Admin View of All Users

Route: /view-all-users

Template: all_users.html

Displays a list of all users and their respective plans. Rendered with Jinja2 using a {% for user in users %} loop, it allows administrators to see original and updated plans clearly in a structured table format.

6. Web: View all users & their plans

```
@router.get("/view-all-users", response_class=HTMLResponse)
```

```
def view_all_users(request: Request):
```

```
    db = SessionLocal()
```

```
    users = db.query(User).all()
```

```
    user_data = []
```

```
    for user in users:
```

```
        plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user.id).first()
```

```
        user_data.append({
```

```
            "id": user.id,
```

```
            "name": user.name,
```

```
            "age": user.age,
```

```
            "weight": user.weight,
```

```
            "goal": user.goal,
```

```
            "intensity": user.intensity,
```

```

        "original_plan": plan.original_plan if plan else "N/A",
        "updated_plan": plan.updated_plan if plan and plan.updated_plan else "Not
updated"
    })

db.close()

return templates.TemplateResponse("all_users.html", {
    "request": request,
    "users": user_data
})

```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in `routes.py`. Each route is linked to the corresponding function developed in Activity 2.1.

Process User Input

HTML Form: Created in `index.html`

Users enter: `username`, `user_id`, `age`, `weight`, `goal`, and `intensity`

FastAPI captures inputs using `Form(...)` parameters

Pydantic schemas (`UserInput`, `FeedbackRequest`) are used to validate structure

Integrate Gemini API Calls

Each route handler connects directly to the respective Gemini-powered function:

`/generate-workout` →

`generate_workout_gemini()` for workout plan

`generate_nutrition_tip_with_flash()` for tip

`/submit-feedback` →

Retrieves plan

Sends original + feedback to `update_workout_plan()`

All AI-generated responses are returned cleanly and injected into the frontend templates for display.